Senior Design Outline (v2): Multi-Provider GPU Delivery via Futures

Group 41 Senior Design

January 2026

Contents

1 A Preliminary Setting **4**

2 Platform Details **4**
2.1 Canonical Concepts & Fixed Parameters (v2) . . . 4
2.2 Market Structure: Tiers, Vouchers, Indices . . . 6
2.2.1 Capability Tiers . . . 6
2.2.2 Normalization & Payment . . . 6
2.2.3 Liquidity & Indices . . . 6
2.3 Entities & Data Models (v2) . . . 6
2.3.1 PackageDescriptor v2 (buyer-supplied) . . . 6
2.3.2 CollectiveSessionDescriptor (new, buyer-supplied) . . . 7
2.3.3 Voucher1155 (financial layer) . . . 8
2.3.4 Receipt (per accepted/rejected package) . . . 8
2.3.5 SessionReceipt (new, optional aggregation) . . . 8
2.4 End-to-End Operating Timeline (Fixed) . . . 8
2.4.1 Day- to Week-Ahead (Buyer & Market) . . . 8
2.4.2 Day-Ahead to Gate Close (Providers & Scheduler) . . . 9
2.4.3 Prepare ($T_0 - 3$m to T_0) . . . 9
2.4.4 Execute (T_0 to deadline) . . . 9
2.4.5 Verification (parallel to execution) . . . 9
2.4.6 Disputes (triggered, package-scoped) . . . 10
2.4.7 Settlement & Anchoring ($T_0 + \varepsilon$) . . . 10
2.4.8 Egress ($T_0 + \varepsilon + 1$) . . . 10
2.5 Scheduling & Eligibility Rules (Hard) . . . 10
2.6 Security, Privacy, & Safety (Operational Defaults) . . . 10
2.7 SLOs, Metrics, & Penalties . . . 11
2.8 API Surfaces (Minimal, Stable) . . . 11
2.8.1 Buyer . . . 11
2.8.2 Provider . . . 11
2.8.3 QC Oracle & Settlement . . . 12
2.9 State Machines (Authoritative) . . . 12
2.9.1 Package State . . . 12
2.9.2 Voucher State . . . 12
2.9.3 Collective Session State (new) . . . 12

2.10 Fixed Non-Goals (v2) . . . 12
2.11 Buyer Guidance (Sizing & Multi-Provider Use) . . . 13
2.12 Compliance Checklist (Go/No-Go) . . . 13

3 A Sample Timeline **13**
3.1 Sample End-to-End Timeline (Detailed Case) . . . 13

4 Reference Workload: GPU-Accelerated CADO-NFS (v2) **14**
4.1 What CADO-NFS does, in plain terms . . . 14
4.2 Why this workload fits our goals . . . 14
4.3 Mathematical skeleton (minimal but sufficient) . . . 14
4.4 How we run CADO-NFS on GPUs in v2 (multi-provider) . . . 14
4.5 Expected artifacts and reproducibility hooks . . . 14

5 The Main Technical Problems (v2) **15**
5.1 P1 — Package Sizing for Milestones Under Heterogeneous Devices (unchanged spine) 15
5.2 P2 — Cross-Arch Canonicalization & Equivalence That Survives Duplication . . . 15
5.3 P3 — Multi-Provider Placement With Gang Constraints (new in v2) . . . 15
5.4 P4 — Normalization & Anti-Gaming Under Collective SLA . . . 16
5.5 P5 — Verification for Collective Computations With Certificates . . . 16
5.6 P6 — Index Integrity & Futures-to-Delivery Correctness (unchanged spine, harder adversary) . . . 16

6 Engineering Design Tracks (Practice-Oriented, Platform-Critical) **16**
6.1 D1 — On-Chain Anchoring & Settlement Wiring (Voucher1155, ReceiptRegistry, Settlement) . . . 16
6.2 D2 — Decentralized Provider Provisioning & Identity (Nomination, Capability, Sybil Resistance) . . . 17
6.3 D3 — Artifact Relay & Distribution (HTTP-Only, Content Integrity, Low T_1) . . . 17
6.4 D4 — Scheduler & Wave Controller (Gate Close, Placement, Reassignment, Standby) 17
6.5 D5 — QC Oracle Network & Neutral Pool (Duplication, Spot, Disputes) . . . 17
6.6 D6 — Canonicalization & Content Hashing Libraries (Per Output Schema) . . . 17
6.7 D7 — Buyer Packager CLI & Preflight (Sizing Advisor, Eligibility, Provenance) . . . 17
6.8 D8 — Provider Runtime & Sandbox (Ingress/Egress, Telemetry, Safety) . . . 18
6.9 D9 — Hourly Tier Indices & Secondary Market Plumbing . . . 18
6.10 D10 — Observability & Receipts (Live Views, Audit Bundles) . . . 18
6.11 D11 — Neutral Dispute Workflow & Evidence Packaging . . . 18
6.12 D12 — Regionalization & Multi-Region Keys (Latency, Locality, Compliance) . . . 18
6.13 D13 — Disaster Recovery & Event Sourcing (Idempotency, Replays, Audits) . . . 18
6.14 D14 — Security & Supply Chain (Image Admission, SBOM, CVE Gates) . . . 19

7 Operational Walkthroughs (Buyer & Providers) **19**
7.1 Buyer Walkthrough A: Standard SLA (Many Independent Packages Across Many Providers) . . . 19
7.2 Buyer Walkthrough B: Collective SLA (Multi-Provider GPU Session) . . . 23
7.3 Provider Walkthrough: Seller of Compute (Standard SLA) . . . 25
7.4 Provider Walkthrough: Seller of Compute (Collective SLA, Multi-Provider Session) . 26
7.5 Multi-Provider Example: "One Buyer, Many Sellers" (Concrete End-to-End) . . . 28

8 Testbed Execution Plan (4× RTX 3060): How We Test v2 End-to-End **29**
8.1 Testing Principles (What Must Be True) . . . 29
8.2 Testbed Parameterization (RTX 3060 Defaults) . . . 29
8.3 Local Multi-Provider Emulation (Required) . . . 30
8.4 Software Components Needed for the Testbed . . . 30
8.5 Test A: Standard SLA Smoke Test (Independent Packages) . . . 30
8.5.1 A1. Setup (One-Time per Test Run) . . . 31
8.5.2 A2. Provider Actions (Per Window) . . . 31
8.5.3 A3. QC & Settlement Checks (Assertions) . . . 31
8.5.4 A4. Pass/Fail Criteria . . . 32
8.6 Test B: Collective SLA Smoke Test (4-Rank Session) . . . 32
8.6.1 B1. Session Configuration . . . 32
8.6.2 B2. Overlay Emulation (Optional but Recommended) . . . 32
8.6.3 B3. Execution & Assertions . . . 32
8.6.4 B4. Pass/Fail Criteria . . . 33
8.7 Test C: CADO-NFS Demo (Scaled for RTX 3060) . . . 33
8.7.1 C1. Scaled Parameters . . . 33
8.7.2 C2. Demonstration Checklist . . . 33
8.8 Negative Tests (Required) . . . 34
8.9 Artifacts to Collect for the Final Report (What We Show) . . . 34

9 Conclusion **34**

10 Glossary & Concept Dictionary (Read This First) **34**
10.1 How to use this glossary . . . 35
10.2 Core Building Blocks . . . 35
10.3 Execution Objects . . . 38
10.4 Verification & Trust . . . 39
10.5 Connectivity & Collective Terms (v2 additions) . . . 42

1 A Preliminary Setting

Alice needs a large amount of GPU compute next week and wants *cost certainty* and *delivery certainty*. On this platform, compute is a traded commodity measured in normalized GPU-hours (NGH) and organized by **hour, region, service level, and capability tier** (e.g., "Tier 1, US–East, 14:00–15:00").

Ahead of time, she can hedge pricing risk by buying **physically deliverable futures/forwards** that settle into vouchers for the exact hour she cares about, or she can buy vouchers directly in a secondary market. Liquidity comes from more than providers: any voucher holder can sell.

As the window approaches, the market hands off to delivery. Day-ahead (or at contract expiry), Alice's positions become vouchers; she deposits vouchers against her job. Providers separately nominate how many NGH they can deliver for that hour. Five minutes before the start, the marketplace gate closes and placements are finalized.

Version 2 goal. We keep the v1 financial layer (vouchers, indices, settlement) and the v1 package spine (sealed work units, receipts, on-chain anchoring), but we add one capability: **multi-provider GPU collectives with guaranteed connectivity**. This enables a single logical job to use GPUs drawn from multiple independent providers, under a bounded time window, with controlled peer-to-peer networking.

Demo objective. Demonstrate that GPU-accelerated CADO-NFS runs end-to-end using GPUs sourced from multiple providers, and that those GPUs are delivered using futures contracts that mint vouchers on our marketplace. The design should generalize to any GPU job that is parallel across many GPUs, and to a subset of tightly-coupled GPU jobs that require cross-node communication.

2 Platform Details

The goal is to fix the structure and operation of CoreIndex's exchange platform v2 that a buyer uses to acquire compute and run work on decentralized third-party providers. It covers: (i) the financial marketplace (hedging and vouchers), (ii) the physical delivery of compute (scheduling, execution, verification), (iii) connectivity guarantees for multi-provider GPU collectives, and (iv) mechanical settlement that pays or slashes providers.

2.1 Canonical Concepts & Fixed Parameters (v2)

The items below are the shared language of the platform. Each term is defined in plain words first, then pinned to a precise, fixed value so readers can reason about behavior without hunting elsewhere.

- **Unit of trade — normalized GPU-hour (NGH).** We measure and price work in normalized GPU-hours. One NGH is the amount of work you would get from one hour on an H100-class data-center GPU under a published benchmark profile. Because real devices differ in speed, raw device time is converted to NGH using a normalization factor (a performance ratio against the H100 baseline). All accounting, receipts, and indices are expressed in NGH.

- **Hour windows.** The fundamental delivery window is one hour. The start of the hour is denoted T_0 and the end is $T_0 + 1\text{h}$. Futures/forwards settle into vouchers keyed to the hour.

- **Lots and package targeting — 10 NGH, multi-provider eligible.** The market's lot size is 10 NGH. Buyers therefore aim to create sealed packages that each consume about ~ 10 NGH of work (anything in the 5–15 NGH range is acceptable). A package is a self-contained unit

(container image, command, inputs, declared output schema) that runs on an assigned provider resource note. In v2, a *job* may span multiple providers by submitting many packages, and may optionally declare a *collective session* for packages that require peer connectivity.

- **Service levels (SLA classes).** v2 defines two service classes that share the same hour window but differ in connectivity:
  - **Standard SLA (default).** Same user-visible behavior as v1: a bounded prepare window of ≤ 3 minutes immediately before T_0; a hard deadline for the prepare window (not best-effort); and a p95 target for timeliness.
  - **Collective SLA (new).** Includes Standard behavior, plus guaranteed in-session connectivity among a declared set of workers ("gang") possibly spanning multiple providers. Connectivity is delivered via a session-scoped overlay network with strict membership ACLs. Collective SLA adds network SLOs (see §2.7).
- **Prepare window.** Prepare runs from $T_0 - 3$m to T_0. It does *not* start billing. Providers must: (i) preflight device/driver, disk/network; (ii) pull the exact image_digest (or serve from cache); (iii) prefetch first input chunk; and (iv) attest ready within the window. Under Collective SLA, providers must additionally (v) join the overlay session and complete key exchange and routing setup before T_0.
- **Determinism modes — bit-exact or FP-tolerant.** Buyers declare how strictly outputs must match across runs. In bit-exact mode, outputs must be identical byte-for-byte. In FP-tolerant mode, floating differences are allowed up to fixed thresholds: `rel_tol` $= 10^{-4}$ and `max_ulp` $= 2$. Outputs are canonicalized before comparison.
- **Milestones — early signal and controlled release.** Two checkpoints structure each run: (1) first outputs must appear within ≤ 2 minutes of T_0; and (2) the scheduler may start a second wave at $T_0 + 2$m once initial throughput is confirmed. These milestone times are fixed for v2. For Collective SLA, "first outputs" may be replaced by "first heartbeat + first batch" for long-running kernels, but the time bound is unchanged.
- **Quality control policy — lightweight, always-on sampling.** Three checks run continuously with low overhead: duplication of 5% of packages, canaries of about 1% of each package, and spot re-execution of 0.5% of items from non-duplicated packages on a neutral pool. In v2, workloads may additionally declare an **algebraic certificate** verifier (e.g., a kernel vector check) that is used as a supplemental acceptance signal.
- **Acceptance rule — package-scoped and binary.** Each package is judged on its own. It is accepted if canaries pass, and (when duplicated) results agree under the declared comparison rule, and spot checks pass. Otherwise it is rejected. This boundary keeps settlement mechanical and audit trails simple.
- **Collective session invariant (new).** A collective session does not replace packages; it groups a set of packages that need mutual connectivity. Session membership is fixed at gate close. Session networking is restricted to member-to-member traffic and expires at the hour boundary.

2.2 Market Structure: Tiers, Vouchers, Indices

2.2.1 Capability Tiers

- **Tier 1:** H100/H200/B200/MI300X-class ($\geq$ 80GB HBM, high tensor throughput, NVLink/IF available)
- **Tier 2:** A100/L40S/MI250-class (40–80GB, mid/high tensor throughput)
- **Tier 3:** Prosumer/legacy (e.g., 4090/A6000/L20), no NVLink requirement

Vouchers and indices are tiered by $(\texttt{region}, \texttt{hour}, \texttt{SLA}, \texttt{tier})$.

2.2.2 Normalization & Payment

For each output schema, the platform publishes a microbenchmark baseline (e.g., tokens/s, images/s, steps/s). Each provider reports a score; the normalization multiplier

$$q = \frac{\texttt{baseline_score}}{\texttt{device_score}}$$

(capped within tier) yields

$$\texttt{paid_ngh} = \texttt{gpu_time_hours_raw} \times q$$

per accepted package. Normalization never overrides SLA feasibility.

2.2.3 Liquidity & Indices

Any voucher holder may buy or sell; providers and traders supply depth. The platform publishes hourly spot indices per $(\texttt{region}, \texttt{tier}, \texttt{SLA})$, used for spot settlement when no voucher is deposited or to value imbalances. Indices are computed using only accepted work and include manipulation guardrails (see P6 / D9).

2.3 Entities & Data Models (v2)

2.3.1 PackageDescriptor v2 (buyer-supplied)

Listing 1: PackageDescriptor v2 (JSON-like)

```
{
  "package_id": "pkg000123",
  "image_digest": "oci://registry/path@sha256:...",
  "command": ["python", "worker.py", "--arg", "val"],
  "input_manifest_uri": "https://bucket/manifest.parquet",
  "input_merkle_root": "0xabc123...",
  "expected_items": 100000,

  "determinism": "bitexact|fp_tolerant",
  "tolerance": { "rel_tol": 1e-4, "max_ulp": 2 },

  "canaries_uri": "https://bucket/canaries.parquet",
  "output_schema": "cado_relations@1|cado_linear_algebra@1|embeddings@1|...",
  "output_uri_prefix": "s3://alice/job123/pkg000123/",
```

```
  "requires": {
    "tier": "T1|T2|T3",
    "min_vram_gb": 80,
    "nvlink": false,
    "driver_stack": "cuda12.4+cudnn9",
    "disk_read_mbps": 500,
    "arch": ["sm90"] // optional; recommended for bitexact
  },

  "collective": {
    "mode": "none|session_member|session_leader",
    "session_id": "sess001", // required if mode != none
    "rank": 0, // required if session_member/leader
    "world_size": 64, // fixed at gate close
    "net_profile": "collective_v1" // references SLA network SLO bundle
  },

  "prefers": { "tier": "T1" }
}
```

2.3.2 CollectiveSessionDescriptor (new, buyer-supplied)

A collective session describes a set of packages that will communicate over the overlay. It is created alongside the job and finalized at gate close.

Listing 2: CollectiveSessionDescriptor (JSON-like)

```
{
  "session_id": "sess001",
  "job_id": "job123",
  "sla_class": "Collective",
  "region": "us-east",
  "hour": "2026-02-03T14:00:00Z/2026-02-03T15:00:00Z",

  "world_size": 64,
  "membership": ["pkg000201", "...", "pkg000264"],

  "overlay": {
    "handshake": "mTLS+ephemeral_keys",
    "allowed_ports": [443, 8443],
    "acl": "members_only",
    "egress_policy": "deny_by_default_allowlist_storage"
  },

  "net_slo": {
    "max_start_skew_sec_p95": 10,
    "min_bw_mbps_p50": 2000,
    "max_rtt_ms_p95": 80
  }
}
```

2.3.3 Voucher1155 (financial layer)

- **Key:** (`region`, `hour`, `SLA`, `tier`).
- **States:** Minted (at contract expiry/day-ahead), Deposited (escrowed to job), Consumed (1:1 with accepted NGH).

2.3.4 Receipt (per accepted/rejected package)

Listing 3: Receipt v2 (JSON-like)

```
{
  "package_id": "pkg000123",
  "provider_fingerprint": "prov9f3b...",
  "input_root": "0xabc123...",
  "output_root": "0xdef456...",
  "item_count": 100000,
  "wall_time_sec": 3470,

  "gpu_time_hours_raw": 9.8,
  "q_multiplier": 1.07,
  "paid_ngh": 10.486,

  "logs_uri": "https://relay/logs/prov9f3b.../pkg000123",

  "qc": {
    "duplicated": true,
    "duplicate_match": "bitexact|within_tolerance|mismatch",
    "canaries_passed": true,
    "spot_reexec_passed": true,
    "certificate_verified": false
  },

  "collective": {
    "session_id": "sess001",
    "rank": 0,
    "net_telemetry_root": "0xnet..." // optional; present for Collective SLA packages
  }
}
```

2.3.5 SessionReceipt (new, optional aggregation)

For Collective SLA, the platform may additionally emit a session-scoped receipt with aggregated network and liveness metrics (used for SLO reporting and slashing triggers), anchored on-chain alongside package receipts.

2.4 End-to-End Operating Timeline (Fixed)

2.4.1 Day- to Week-Ahead (Buyer & Market)

1. **Plan & Hedge:** Buyer selects windows by (`region`, `hour`, `SLA`, `tier`) and acquires exposure via physically deliverable forwards/futures or secondary vouchers. Any holder may resell.

2. **Package:** Buyer produces a package index of 100–1000 sealed packages sized ~ 10 NGH.
3. **Collective declaration (optional):** Buyer defines one or more CollectiveSessionDescriptors for packages that need peer connectivity, including world_size and net_SLO bundle.
4. **Artifacts:** Upload manifests/canaries/models; register large artifacts with Artifact Relay (HTTP shards + checksums).
5. **Create Job:** Submit job_id, package index, determinism/tolerance, output schema; opt to deposit vouchers or clear at spot.

2.4.2 Day-Ahead to Gate Close (Providers & Scheduler)

6. **Mint & Deposit:** At contract expiry/day-ahead, clearinghouse mints vouchers; buyer deposits against job_id.
7. **Nomination:** Providers nominate deliverable NGH by window and tier; scheduler computes placement plan honoring requires/prefers and milestone feasibility. For Collective SLA sessions, scheduler additionally enforces network feasibility constraints.
8. **Gate Close ($T_0 - 5$m):** Finalize mapping: package $\rightarrow$ provider; finalize collective session membership and routing; publish prepare tasks.

2.4.3 Prepare ($T_0 - 3$m to T_0)

Providers must: (i) preflight device/driver, disk/network; (ii) pull exact image_digest; (iii) prefetch first input chunk; (iv) attest ready via `/lots/{id}/prepare_ready` within ≤ 3 minutes. SLA clock starts at T_0, not before. For Collective SLA: (v) join overlay, complete key exchange, validate ACL membership, and report `/sessions/{sid}/ready`.

2.4.4 Execute (T_0 to deadline)

- **Wave-1 & Headroom:** launch enough packages to meet deadline with $\sim 15\%$ headroom.
- **Wave-2:** start at $T_0 + 2$m after throughput confirmation.
- **Milestones:** each package emits first outputs ≤ 2 minutes; miss $\Rightarrow$ cancel & whole-package reassignment to standby.
- **Failure:** any provider fault $\Rightarrow$ cancel & whole-package reassignment; platform never edits/splits package contents.
- **Collective sessions:** membership fixed; overlay traffic allowed only among session members; outside egress remains allowlisted.

2.4.5 Verification (parallel to execution)

- **Duplication:** 5% of packages run on a second provider (within tier; within arch if bit-exact).
- **Canaries:** $\sim 1\%$ known-answer items per package.
- **Spot re-exec:** 0.5% of items from each non-duplicated package on a neutral pool.

- **Decision:** apply fixed acceptance rule; rejected packages are unpaid, slashed, and immediately reassigned.
- **Certificate checks (optional):** for workloads with algebraic certificates (e.g., CADO kernel vector validity), validate certificates on neutral pool; record in QC evidence.

2.4.6 Disputes (triggered, package-scoped)

On dup mismatch, canary fail, or spot fail: re-exec 10% of that package's items on a neutral pool. If mismatch > tolerance: mark provider at fault, reject & slash (forfeit payout + imbalance fee), and reassign. Supplier-of-last-resort capacity, funded by penalties, preserves buyer deadlines.

2.4.7 Settlement & Anchoring ($T_0 + \varepsilon$)

QC Oracle computes two Merkle roots: `receipt_root` (over all package receipts) and `qc_root` (over QC decisions/events). ReceiptRegistry stores both; Settlement pays per accepted package using `paid_ngh`. Deposited vouchers are consumed 1:1 with accepted NGH; any shortfall spot-settles at the published hourly tier index.

2.4.8 Egress ($T_0 + \varepsilon + 1$)

Buyer downloads outputs (per-package shards) and the receipt bundle (receipts + QC outcomes + Merkle proofs). Anyone can verify outputs/inputs against on-chain roots or re-run a package locally (same image & manifest).

2.5 Scheduling & Eligibility Rules (Hard)

- **Eligibility:** provider must satisfy all `requires.*` fields; bit-exact packages should pin `arch` to avoid drift across device families.
- **Feasibility check:** placement pre-check ensures milestone feasibility given device performance and package sizing; normalization does not bypass SLA checks.
- **Mixing across devices:** allowed across packages. A job may route different packages to different tiers if both satisfy requires.
- **No platform-side splitting:** if a package is oversized for the chosen tier, it is ineligible; platform never re-chunks.
- **Collective feasibility (new):** for Collective SLA sessions, the scheduler must satisfy: (i) all members placed by $T_0 - 5$m, (ii) overlay membership fixed at gate close, (iii) net_SLO feasibility under measured inter-provider connectivity.

2.6 Security, Privacy, & Safety (Operational Defaults)

- **Sandboxing:** egress blocked by default; allowlisted storage endpoints; per-package network/disk quotas.
- **Artifact integrity:** all Relay shards carry published checksums; container image pinned by digest.

- **Auditability:** per-package logs URI; roots anchored on-chain; receipt bundle includes Merkle proofs.
- **Collective overlay (new):** session-scoped encrypted tunnels, members-only ACL, deny-by-default routing, packet accounting for SLO enforcement, and automatic teardown at hour end.

2.7 SLOs, Metrics, & Penalties

Metric / SLO	Definition	Target
Prepare readiness	Provider attests within $\leq$ 3m before T_0	Hard
First-output latency	$\leq$ 2m from T_0 per running package	p95
Wave-2 start	$T_0 + 2$m after throughput confirmation	Fixed
QC rates	Dup 5%; canaries $\sim$1%; spot 0.5% items	Fixed
Settlement time	Post/anchor at $T_0 + \varepsilon$; pay/slash atomic per package	Fixed
Penalty funding	Slashed imbalance fees fund supplier-of-last-resort capacity	Fixed
Collective start skew	Max rank-to-rank start skew (heartbeat)	p95 $\leq$ 10s
Collective bandwidth	Median in-session throughput	p50 $\geq$ 2 Gbps
Collective RTT	In-session RTT between members	p95 $\leq$ 80 ms

Penalties mirror v1: failed milestones and QC failures trigger package reassignment and slashing. Collective SLA adds: failure to meet declared net_SLO beyond tolerance triggers partial slashing or reputation penalties, funding standby and neutral verification pools.

2.8 API Surfaces (Minimal, Stable)

2.8.1 Buyer

- `POST /jobs`: submit job_id, package index, determinism/tolerance, output schema.
- `POST /sessions`: create CollectiveSessionDescriptor(s) (optional).
- `POST /vouchers/deposit`: escrow vouchers to job_id.
- `GET /jobs/{id}/feasibility`: earliest start, required NGH, voucher gaps.
- `GET /jobs/{id}/receipts`: receipt bundle on completion.

2.8.2 Provider

- `POST /nominations`: declare per (region, hour, tier, SLA).
- `POST /lots/{id}/prepare_ready`: attest readiness.
- `POST /sessions/{sid}/ready`: attest overlay joined (Collective SLA only).
- `POST /lots/{id}/result`: submit output root, item count, wall time, raw GPU time, logs URI.

2.8.3 QC Oracle & Settlement

- `POST /qc/duplicate`: submit duplicate comparison outcome.
- `POST /qc/spot`: submit spot re-exec result.
- `POST /qc/cert`: submit certificate verification outcome (optional).
- `POST /settlement/anchor`: publish receipt_root and qc_root.
- `POST /settlement/pay`: pay/slash per accepted/rejected package using paid_ngh.

2.9 State Machines (Authoritative)

2.9.1 Package State

State Transitions
Created → Placed (at Gate Close)
Placed → Prepared (provider attests ready)
Prepared → Running (T_0 start)
Running → Accepted (QC pass) — Rejected (QC fail) — Reassigned (milestone miss/fault)
Accepted → Settled (paid)
Rejected → Reassigned (new provider)

2.9.2 Voucher State

State Transitions
Minted → Deposited (escrow to job) — Traded (secondary market)
Deposited → Consumed (1:1 with accepted NGH)

2.9.3 Collective Session State (new)

State Transitions
Declared → Finalized (at Gate Close; membership fixed)
Finalized → Ready (all members attest overlay joined)
Ready → Active (T_0 start)
Active → Expired (at $T_0 + 1$h)
Active → Degraded (net_SLO violation)

2.10 Fixed Non-Goals (v2)

- **No platform-side algorithmic parallelization.** Buyer owns decomposition into packages and/or collective roles.
- **No arbitrary internet egress.** Egress is allowlisted; collective overlay is members-only and session-scoped.
- **No unbounded collectives.** Collective SLA is limited to declared session sizes and fixed net_SLO bundles; the platform does not promise HPC-grade low-latency across the public internet.

2.11 Buyer Guidance (Sizing & Multi-Provider Use)

- Size packages for the slowest tier you permit in `requires.tier`; milestones are device- and size-coupled.
- Use bit-exact with pinned `arch` when you need byte-for-byte reproducibility across duplicates.
- Use FP-tolerant when heterogeneity is expected and semantic equivalence suffices.
- Use Collective SLA only for packages that truly need peer connectivity (e.g., distributed reductions or model-parallel shards). For embarrassingly parallel stages, prefer Standard SLA across many providers.

2.12 Compliance Checklist (Go/No-Go)

- Package index loads; all packages in [5, 15] NGH and immutable.
- `requires.*` satisfied by candidate placements; feasibility check passes milestones.
- Prepare attestations within $\leq$ 3m for all lots; image digests match.
- Collective sessions (if any) finalized at gate close; overlay joins complete before T_0.
- QC sample rates achieved; disputes resolved within policy; roots anchored on-chain.
- Voucher consumption equals accepted NGH; provider payouts match paid_ngh.

3 A Sample Timeline

3.1 Sample End-to-End Timeline (Detailed Case)

Case Parameters (Fixed). Alice runs a v2 demo to factor a composite integer N using GPU-accelerated CADO-NFS. She targets a single window: Tuesday, 14:00–15:00 (US–East), **Collective SLA**, Tier 1. She submits: (i) many Standard packages for sieving across multiple providers, and (ii) one Collective session for distributed linear algebra across providers.

Time	Phase	Event (normative)
$T - 7$d	Plan/Hedge	Alice buys physically deliverable exposure for 1,600 NGH (Tier 1, US–East, 14:00
$T - 3$d	Package	140 sealed packages produced: 120 sieve-interval packages (Standard), 20 LA-wor
$T - 3$d	Artifacts	Manifests/canaries/params uploaded; large inputs pinned to Artifact Relay
$T - 1$d	Job Create	Alice submits job_id, package index, determinism, session descriptor; deposits vou
$T - 24$h	Mint/Deposit	Clearinghouse mints vouchers; Alice deposits 1,600 NGH to job_id
$T - 24$h	Nomination	Providers nominate Tier 1 capacity; scheduler drafts placement + session membe
$T - 5$m	Gate Close	Package→provider map fixed; collective session membership finalized
$T - 3$m	Prepare	Providers preflight, pull image, prefetch; Collective members join overlay
T_0	Execute	Wave-1 starts; sieve packages begin; collective LA workers begin heartbeats
$T_0 + 2$m	Wave-2	Wave-2 starts after throughput confirm
$T_0 \rightarrow$deadline	QC	Dup/canary/spot + certificate checks for LA outputs
$T_0 + \varepsilon$	Settle	QC Oracle anchors roots; Settlement pays accepted; vouchers consumed 1:1
$T_0 + \varepsilon + 1$	Egress	Alice downloads relation shards, LA artifacts, and receipt bundle

4 Reference Workload: GPU-Accelerated CADO-NFS (v2)

This section fixes a single, concrete workload—CADO-NFS, the Number Field Sieve implementation—for platform shakeout. We explain the pipeline, the mathematics governing cost/correctness, and how its phases map onto heterogeneous GPUs *including* a multi-provider collective phase in v2. We choose CADO-NFS because it is public, deterministic, and offers strong correctness certificates.

4.1 What CADO-NFS does, in plain terms

Given a composite integer N, NFS finds a non-trivial factor by: (i) polynomial selection; (ii) relation collection (sieving); (iii) filtering/recombination to form a sparse matrix; (iv) linear algebra to find a kernel vector over $\mathbb{F}_2$ (or $\mathbb{F}_p$); and (v) square-root to yield a factor via gcd.

4.2 Why this workload fits our goals

Relation collection is embarrassingly parallel: the search space decomposes into independent "sieve intervals," each runnable in isolation on a single GPU with no synchronization. In v2, we additionally demonstrate a multi-provider collective by distributing the sparse linear algebra step across GPUs from multiple providers, using the Collective SLA overlay.

4.3 Mathematical skeleton (minimal but sufficient)

Fix two coprime polynomials $f, g \in \mathbb{Z}[x]$ of small degree with $\mathrm{Res}(f, g)$ comparable to N. For integers a, b with $b \neq 0$, define rational and algebraic values $R(a, b)$ and $A(a, b)$. A relation is a pair (a, b) for which both sides factor over pre-fixed factor bases up to a bound B. After collecting enough relations, we construct a huge sparse matrix $M \in \{0, 1\}^{m \times n}$. A non-zero kernel vector v over $\mathbb{F}_2$ satisfies $Mv = 0$ and yields a congruence of squares modulo N.

Certificate property. Given M and a candidate v, correctness can be checked by verifying $Mv = 0$ (over $\mathbb{F}_2$) and that the final gcd produces a non-trivial factor. This gives an unusually strong verification hook for a decentralized platform.

4.4 How we run CADO-NFS on GPUs in v2 (multi-provider)

1. **Relation collection (Standard packages).** Each sieve interval is a sealed package targeting one GPU. It emits relation records and partials. Outputs are serialized in canonical order so correct executions produce identical hashes.

2. **Filtering (bounded reduction).** Filtering and clique recombination run as a bounded-scope reduction that can execute either: (i) on one high-memory node (Standard), or (ii) on a small Collective session if memory is sharded.

3. **Linear algebra (Collective session).** We use block Lanczos over $\mathbb{F}_2$ (bit-packed SpMV and XOR reductions). We partition matrix rows across ranks; each rank computes local SpMV and participates in XOR-based reductions over the overlay network. The arithmetic is boolean and can be bit-exact. The final kernel vector v is verified by certificate checks on the neutral pool.

4.5 Expected artifacts and reproducibility hooks

Inputs: target integer N; parameter files; list of sieve intervals with fixed seeds. Intermediate outputs: relation files; large-prime graphs; filtered sparse matrix (bit-packed). Final outputs: kernel

vector(s) over $\mathbb{F}_2$ and the factors of N. Each artifact is accompanied by a content hash; relation files are serialized in canonical order. Collective LA runs checkpoint at deterministic iteration counts and include hashes of Krylov vectors, so resume continues the same trajectory.

5 The Main Technical Problems (v2)

Below are six tightly-scoped, research-grade problem statements. Each one is written to be executable (clear inputs/outputs, invariants, tests) and tied to fixed platform mechanics: tiered NGH, whole-package routing, prepare $\leq$ 3m, first-output $\leq$ 2m, QC sampling, Receipt/QC Merkle roots, voucher consumption 1:1 with accepted NGH, and paid_ngh = raw GPU hours $\times q$. v2 adds one new axis: Collective SLA overlay connectivity across multiple providers.

5.1 P1 — Package Sizing for Milestones Under Heterogeneous Devices (unchanged spine)

Context. Buyer submits sealed packages sized near 10 NGH (acceptable 5–15) that may run on any provider satisfying requires. Platform enforces first-output $\leq$ 120s and hour-deadline completion; misses cause whole-package reassignment.

Technical question. For an output schema s, choose an item count W per package such that when routed to any device meeting `requires.tier`, the probability of meeting both milestones exceeds a target (e.g., 0.99), while W maps to billed NGH in [5,15].

What a solution must deliver. Sizing advisor $W^\star$ with confidence bands; robust calibration; guardrails when allowing slower tiers.

Evaluation. Backtest across Tier 1/2 devices: $\geq$ 95% first-output success and $\geq$ 99% deadline success without reassignment; reassignments reduced by $\geq$ 50% vs naive sizing.

5.2 P2 — Cross-Arch Canonicalization & Equivalence That Survives Duplication

Context. QC duplicates 5% of packages and performs spot re-exec, using bit-exact or FP-tolerant policy. False rejects cause unfair slashing; false accepts enable cheating.

Technical question. Define schema-specific canonicalization $K_s(\cdot)$ and equivalence $E_s(\cdot,\cdot)$ that are stable across: (i) device arch differences, (ii) parallelism-induced ordering differences, and (iii) serializer variability.

Deliverables. Canonicalizers + comparator; proofs/exhaustive tests; adversarial gold corpus.

5.3 P3 — Multi-Provider Placement With Gang Constraints (new in v2)

Context. In v2, a job may declare Collective sessions: a fixed set of packages that must communicate over an overlay. At gate close, membership is fixed; by T_0 all members must be overlay-ready.

Technical question. Given provider nominations and measured inter-provider connectivity, choose placements that satisfy: (i) per-package eligibility/milestones, and (ii) for each session, network feasibility (start skew, RTT, bandwidth) under the declared net_SLO bundle, while minimizing expected reassignment and standby usage.

Deliverables. A placement optimizer with a network-aware constraint model; a feasibility oracle for session graphs; fallback policies (degrade, shrink, or reassign) that preserve hour deadlines.

Evaluation. Replay simulation with synthetic and measured RTT matrices: meet net_SLO p95 targets; reduce collective startup failures by $\geq$ 40% vs naive "place anywhere".

5.4 P4 — Normalization & Anti-Gaming Under Collective SLA

Context. Providers report benchmark scores to compute q; they may attempt to game measurements. Collective SLA adds a new surface: network and coordination overhead.

Technical question. Design a normalization policy and telemetry scheme that: (i) credits compute fairly (not I/O or idle), (ii) is robust to provider manipulation, and (iii) does not penalize honest collective coordination beyond declared SLOs.

Deliverables. Signed measurement reports; anti-cheat detectors; caps within tier; audit methodology.

5.5 P5 — Verification for Collective Computations With Certificates

Context. Duplication-based QC is expensive for tightly-coupled collectives. Some workloads admit algebraic certificates (CADO does).

Technical question. Define a verification policy that preserves the package-scoped acceptance spine while allowing a session to be accepted based on certificate checks (e.g., verify $Mv = 0$ and factor correctness), plus lightweight sampling for logs/telemetry integrity.

Deliverables. Certificate verifier library; integration into QC Oracle; rules for when certificate overrides duplication.

5.6 P6 — Index Integrity & Futures-to-Delivery Correctness (unchanged spine, harder adversary)

Context. Spot settlement fairness and hedging UX depend on robust indices and liquid trading. Multi-provider execution raises the importance of clean linkage between voucher keys and accepted NGH.

Technical question. Specify voucher–package linkage, idempotent settlement, and index methodology (VWAP per key tuple) with guardrails that bound manipulation to $< \gamma$ given li

6 Engineering Design Tracks (Practice-Oriented, Platform-Critical)

This section enumerates the build-level problems that arise when turning the fixed marketplace design into a working system. Each track states the objective, why it emerges, key design choices, and concrete readiness tests.

6.1 D1 — On-Chain Anchoring & Settlement Wiring (Voucher1155, ReceiptRegistry, Settlement)

Objective. Anchor receipt_root and qc_root on-chain and settle paid_ngh per accepted package while consuming vouchers 1:1 for the correct $(\texttt{region}, \texttt{hour}, \texttt{SLA}, \texttt{tier})$.

Key choices. Contract set; identifier binding; exactly-once semantics; finality model; gas budget.

Readiness. Model-check invariants (no double-consume, no orphaned acceptance); reorg simulation; settlement latency/fee bounds.

6.2 D2 — Decentralized Provider Provisioning & Identity (Nomination, Capability, Sybil Resistance)

Objective. Let any provider join, nominate NGH per window, be schedulable if satisfying requires; ensure stable provider_fingerprint.

v2 addition. Providers optionally attest overlay capability (NAT traversal, ports, bandwidth class).

Readiness. Attestation replay resistance; capability drift detection; simulated Sybil unable to reset reputation.

6.3 D3 — Artifact Relay & Distribution (HTTP-Only, Content Integrity, Low T_1)

Objective. Serve large images/checkpoints/manifests over HTTP with strong integrity and predictable first-output $\leq$ 2m.

v2 note. Collective overlay does not replace Relay; it is not used for artifact distribution.

Readiness. Cold-start T_1 improvement; corruption probability bounds; cache hit-rate targets.

6.4 D4 — Scheduler & Wave Controller (Gate Close, Placement, Reassignment, Standby)

Objective. Compute package→provider assignments that hit milestones and deadlines with minimal reassignment; size Wave-1 headroom and Wave-2 trigger; maintain standby share.

v2 addition. Gang scheduling and network feasibility constraints for collective sessions; session readiness tracking.

Readiness. Offline replay reduces misses; unnecessary reassignments $< 2\%$; collective start-skew p95 meets target.

6.5 D5 — QC Oracle Network & Neutral Pool (Duplication, Spot, Disputes)

Objective. Operate validators that run duplicates, spot re-exec, and dispute re-exec on an independent neutral pool; publish decisions into qc_root.

v2 addition. Certificate checks (e.g., CADO $Mv = 0$ verification) and session telemetry audits.

Readiness. Consistent rulings; reconstructable audit trail; overhead within budget.

6.6 D6 — Canonicalization & Content Hashing Libraries (Per Output Schema)

Objective. Provide schema-specific libraries that transform outputs into canonical forms with stable hashes and tolerant equality.

v2 addition. Canonicalization for CADO relation records, bit-packed matrices, and kernel vectors.

Readiness. Zero false hash mismatches for semantically identical outputs; cross-arch agreement within tolerance.

6.7 D7 — Buyer Packager CLI & Preflight (Sizing Advisor, Eligibility, Provenance)

Objective. CLI that builds sealed packages, injects canaries, computes manifests/roots, checks requires, and refuses infeasible sizes.

v2 addition. Session builder that assigns ranks/world_size, validates net_SLO bundle, and simulates feasibility.

Readiness. $> 95\%$ infeasibilities caught pre-submit; reproducible descriptors; provenance enforced.

6.8 D8 — Provider Runtime & Sandbox (Ingress/Egress, Telemetry, Safety)

Objective. Hardened agent that pulls images, prefetches inputs, runs packages in a sandbox, enforces allowlisted egress, and reports verifiable telemetry.

v2 addition. Overlay joiner: session-scoped tunnels, members-only ACL, traffic accounting, teardown at hour end.

Readiness. Benign jobs unaffected; red-team exfil blocked; GPU-time estimation error $< 2\%$.

6.9 D9 — Hourly Tier Indices & Secondary Market Plumbing

Objective. Publish per-key VWAP using only accepted NGH; operate a pre-delivery voucher market with manipulation guardrails.

v2 addition. Separate indices per SLA class (Standard vs Collective) if liquidity supports; otherwise publish basis adjustments.

Readiness. Attack sims fail to move index $> \gamma$ without large cost; replayable public calc matches published index.

6.10 D10 — Observability & Receipts (Live Views, Audit Bundles)

Objective. Real-time progress, milestone alerts, and compact receipt bundle with proofs matching on-chain roots.

v2 addition. Session dashboards: start skew, RTT, bandwidth; alerts on degradation.

Readiness. Random audit verifies any package quickly; alerting reduces deadline misses.

6.11 D11 — Neutral Dispute Workflow & Evidence Packaging

Objective. Standardize how mismatches trigger neutral re-exec, thresholds, and evidence packaging for slashing.

v2 addition. For collective disputes: evidence includes overlay telemetry roots and membership proofs.

Readiness. Median dispute resolution under bound; independent reviewer reaches same decision.

6.12 D12 — Regionalization & Multi-Region Keys (Latency, Locality, Compliance)

Objective. Keep vouchers/indices/scheduling region-scoped while allowing global accounts.

v2 addition. Overlay POP strategy per region; deny cross-region sessions by default unless net_SLO allows.

Readiness. Region outages isolated; start-time p95 unaffected by global control-plane issues.

6.13 D13 — Disaster Recovery & Event Sourcing (Idempotency, Replays, Audits)

Objective. Event-sourced ledgers for jobs, placements, QC, and settlement with exactly-once semantics and deterministic replays.

v2 addition. Session events and overlay telemetry anchoring integrated into event stream.
Readiness. Full replay regenerates state byte-identical; chaos restarts lose no events.

6.14 D14 — Security & Supply Chain (Image Admission, SBOM, CVE Gates)

Objective. Enforce container provenance and vulnerability policies at prepare-time without blocking legitimate workloads.

v2 addition. Overlay software supply chain attestation (tunnel binary hashes, config policies).

Readiness. Known-bad images always blocked; false block rate near zero; incident drill latency $< \Delta t$.

7 Operational Walkthroughs (Buyer & Providers)

This section is intentionally procedural: it enumerates *every* action, decision, and option a buyer and providers take for an end-to-end run. It is written as if the platform exposes (i) a web UI (`Market`, `Jobs`, `Sessions`, `Provider Portal`) and (ii) a CLI (`coreindex`) that performs the same operations with explicit flags. All steps below are normative for the v2 demo.

7.1 Buyer Walkthrough A: Standard SLA (Many Independent Packages Across Many Providers)

Goal. Run an embarrassingly-parallel GPU job (e.g., CADO sieving, embeddings, Monte Carlo) using GPUs from multiple providers. No peer-to-peer worker networking is needed.

Buyer prerequisites (one-time).

1. Create an account (wallet address) and bind a payout/charge method for fees.
2. Install CLI: `coreindex` and authenticate (`coreindex auth login`).
3. Configure storage credentials for output egress (S3/GCS/etc.) and allowlist the endpoints in your job profile.
4. (Optional) Configure a container registry account; ensure images are pinned by digest.

Step 1 — Choose product key (window + SLA + tier).

1. Open `Market` (UI) or run: `coreindex market browse`.
2. Select:
   - **Region:** choose one region (default: `us-east`).
   - **Hour:** choose the exact hour window (displayed in your local timezone).
   - **SLA:** choose `Standard`.
   - **Tier:** choose `Tier 1 / Tier 2 / Tier 3`.
3. Decision: **how to acquire** exposure?
   - **Hedge (futures/forwards):** lock price now, mint vouchers at expiry/day-ahead.
   - **Buy vouchers spot:** pay current market price now.

- **Run at spot without vouchers:** settle at index later (not recommended for the demo).

4. Decision: **how much** to buy?
   - Enter target NGH (example: 1,200 NGH for 120 packages at 10 NGH each).
   - UI shows: expected package count at 10 NGH; worst-case buffer (recommend +10–15%).

Step 2 — Acquire exposure (hedge path).

1. In `Market`, click `Trade` → `Physically Deliverable` → `Buy`.
2. Confirm contract details:
   - Key: (region, hour, Standard, tier)
   - Notional: target NGH
   - Expiry: day-ahead or specified settlement time
   - Fees: taker fee + settlement fee
3. Approve transaction (wallet signature). UI shows `Position Created`.
4. At day-ahead/expiry, UI shows `Vouchers Minted` (or CLI: `coreindex vouchers list`).

Step 3 — Build sealed packages (what the buyer decides).

1. Decision: choose **output schema** (e.g., `cado_relations@1`, `embeddings@1`).
2. Decision: choose **determinism mode**:
   - **bitexact** if you can pin architecture and want strict duplication equality.
   - **fp_tolerant** if cross-arch variation is expected.
3. Decision: set **requires**:
   - `tier` (must match voucher tier if you plan to deposit and avoid spot gaps),
   - `min_vram_gb`,
   - `driver_stack`,
   - (optional) `arch` pin if `bitexact`.
4. Generate package manifests:
   - Split work into packages targeting 10 NGH each (acceptable 5–15).
   - Fix seeds and input ordering so outputs are canonical.
   - Compute `input_merkle_root` for each package.
   - Select canaries (∼1% items) and produce `canaries_uri`.
5. Build/pin image:
   - Build container and push to registry.
   - Record immutable `image_digest` (sha256).
6. Run CLI packager:

- `coreindex package build --schema cado_relations@1 --target-ngh 10 --determinism bitexact`
- Output: `packages.jsonl` (one descriptor per package) + local manifest roots report.

7. Run feasibility check (required):
   - `coreindex jobs feasibility --packages packages.jsonl --region us-east --hour 2026-02-03T14`
   - If any package is flagged (too large / first-output risk / requires mismatch), you must rebuild or adjust.

Step 4 — Upload artifacts and register with the Artifact Relay.

1. Upload:
   - `input_manifest` shards (HTTP-accessible) + checksums,
   - canary file,
   - job parameters.
2. Register artifacts:
   - `coreindex relay register --manifest input_manifest.parquet --checksums checksums.json`
3. Confirm allowlist endpoints (storage + relay) in `Jobs` → `Networking Policy`.

Step 5 — Create job and optionally deposit vouchers.

1. Create job:
   - UI: `Jobs` → `Create` → upload `packages.jsonl`.
   - Choose settlement mode:
     - **Voucher-deposited** (recommended for demo): predictable delivery and accounting.
     - **Spot** (allowed): will settle at index for any shortfall.
2. If voucher-deposited:
   - UI: `Vouchers` → `Deposit to Job` → select key tuple and amount.
   - Decide deposit amount: total NGH + buffer.
   - Approve escrow transaction (wallet signature).
3. Confirm job summary screen:
   - package count,
   - total target NGH,
   - window key tuple,
   - artifact endpoints allowlist,
   - QC policy summary.

Step 6 — Observe gate close and prepare.

1. At $T_0 - 10$m, open `Jobs` → `Live`:
   - see nominated supply,
   - see tentative placements (not final).
2. At $T_0 - 5$m (gate close), placements finalize:
   - UI shows each package assigned provider_fingerprint.
   - No changes to package contents are possible.
3. From $T_0 - 3$m to T_0, watch `Prepared` count:
   - packages transition Placed → Prepared when providers attest ready.
   - prepare does *not* bill.

Step 7 — Execution, QC, and completion.

1. At T_0, wave-1 starts. Track:
   - first-output timestamps,
   - running throughput,
   - any reassignments (milestone misses/faults).
2. QC tab shows:
   - duplicated packages and their match status,
   - canary pass/fail,
   - spot re-exec outcomes.
3. When the hour ends, the job finalizes:
   - UI shows receipt_root and qc_root anchored.
   - Vouchers consumed equals accepted NGH.
4. Download outputs and receipt bundle:
   - UI: `Jobs` → `Receipts` → `Download Bundle`.
   - CLI: `coreindex jobs receipts --job job123 --out receipts.tar.gz`.
5. (Optional) Verify locally:
   - Validate Merkle proofs against on-chain roots.
   - Re-run any package using `image_digest` + manifest to audit.

7.2 Buyer Walkthrough B: Collective SLA (Multi-Provider GPU Session)

Goal. Use GPUs from multiple providers *in one coordinated distributed step* that requires peer connectivity (e.g., distributed block Lanczos step in CADO-NFS, or another collective GPU job).

Important. A collective session does not replace packages; it groups packages into a fixed membership with a controlled overlay network.

Step 1 — Choose Collective SLA and session size.

1. In `Market`, select the key tuple with **SLA = Collective**.
2. Decide **world size** (number of workers / ranks). Example: 32 GPUs.
3. Choose **net profile** (a preset SLO bundle): `collective_v1`.
4. Acquire exposure (hedge or spot) exactly as in Walkthrough A, but for the Collective key tuple.

Step 2 — Decide decomposition: what is inside the collective vs outside.

1. Decide which phases are embarrassingly parallel:
   - For CADO-NFS: sieving packages are Standard (no session).
2. Decide which phase is collective:
   - For CADO-NFS: linear algebra worker packages join `session_id`.
3. Decide the session membership list:
   - list of package_ids that participate,
   - rank assignment (0..world_size-1),
   - whether there is a leader (rank 0) for rendezvous and logging.

Step 3 — Build collective packages (extra fields).

1. For each collective worker package:
   - set `collective.mode` to `session_member` (or leader for rank 0),
   - set `session_id`, `rank`, `world_size`, `net_profile`.
2. Decide determinism:
   - For CADO LA over $\mathbb{F}_2$: prefer `bitexact` with pinned arch if possible.
3. Decide checkpointing:
   - fixed iteration checkpoints,
   - hashed state summaries written to storage.
4. Run CLI packager with session config:
   - `coreindex package build --schema cado_linear_algebra@1 --collective session.yaml`

Step 4 — Create the CollectiveSessionDescriptor and submit job.

1. In UI: `Sessions` → `Create`.
2. Select:
   - session_id, job_id,
   - region and hour,
   - world_size,
   - membership list (package ids),
   - net_SLO bundle (start skew, RTT, bandwidth).
3. Confirm overlay policy defaults:
   - members-only ACL,
   - allowed ports,
   - deny-by-default egress with allowlisted storage/relay.
4. Submit session, then submit job with package index.
5. Deposit vouchers against the Collective key tuple (and against Standard tuple for Standard packages if mixed).

Step 5 — Gate close, overlay readiness, and execution.

1. At $T_0 - 5$m: session membership becomes **final**; UI shows provider assignment per rank.
2. At $T_0 - 3$m..T_0: watch two readiness counters:
   - `Prepared` (image pulled + input prefetched),
   - `Session Ready` (overlay join completed for each rank).
3. At T_0: session becomes `Active` if all ranks are ready; otherwise it enters `Degraded` and may reassign missing ranks.
4. During execution: watch session telemetry:
   - start skew,
   - RTT and bandwidth,
   - any rank dropouts or reassignments.

Step 6 — Verification and completion (collective).

1. For CADO LA: upload kernel vector candidate + log hashes; QC runs certificate verification (e.g., verify $Mv = 0$).
2. QC tab shows:
   - certificate verified (yes/no),
   - spot checks on telemetry integrity (if enabled),
   - any disputes and neutral re-exec decisions (if applicable).
3. Settlement consumes Collective vouchers 1:1 with accepted paid_ngh from session member packages.

7.3 Provider Walkthrough: Seller of Compute (Standard SLA)

Goal. A provider supplies GPU capacity into the marketplace and gets paid if packages pass QC.

Provider prerequisites (one-time).

1. Create provider identity and keys (provider_fingerprint).
2. Install `coreindex-provider-agent` and register:
   - hardware inventory (GPU model, VRAM, driver stack),
   - benchmark score for normalization,
   - network allowlist compliance.
3. Pass admission checks:
   - sandbox enforcement,
   - logging/telemetry availability,
   - artifact relay access.

Step 1 — Nominate capacity for a window.

1. Open `Provider Portal` → `Nominations`.
2. Select key tuple:
   - region, hour, SLA=Standard, tier.
3. Decide nomination amount:
   - enter NGH you can reliably deliver.
   - optionally reserve headroom for duplicates/spot jobs.
4. Submit nomination (signed).

Step 2 — Receive placements at gate close.

1. At $T_0 - 5$m, agent receives assigned lots (package descriptors + artifact URIs).
2. Decision: accept or decline? (v2 assumes auto-accept for nominated capacity; declines count against reputation).

Step 3 — Prepare window actions (required).

1. From $T_0 - 3$m to T_0:
   - preflight GPU/driver/disk/network,
   - pull exact `image_digest` (verify hash),
   - prefetch first input chunk,
   - enforce sandbox policies (deny-by-default egress).
2. Call `POST /lots/{id}/prepare_ready` (agent does this automatically).

Step 4 — Execute and report results.

1. At T_0, start container with given command and mount input/output paths.
2. Ensure first outputs are emitted within $\leq$ 2m:
   - for batch outputs: write first shard + update output manifest,
   - for streaming outputs: flush first records and update output root builder.
3. On completion, submit `/lots/{id}/result` including:
   - output_root, item_count,
   - wall time,
   - raw GPU time,
   - logs URI.

Step 5 — Respond to QC/disputes.

1. If chosen as duplicate: run duplicate job as assigned.
2. If dispute triggered:
   - provide requested evidence (logs, telemetry signatures),
   - optionally re-run locally for diagnosis (does not affect settlement).
3. Payment/slashing occurs automatically at settlement.

7.4 Provider Walkthrough: Seller of Compute (Collective SLA, Multi-Provider Session)

Goal. Multiple independent providers each supply some ranks in a collective session and must join an overlay network.

Provider prerequisites (extra for Collective).

1. Enable overlay module in agent:
   - supports session-scoped tunnel creation,
   - can enforce members-only ACL,
   - can measure RTT/bandwidth and report signed telemetry.
2. Pass overlay admission test:
   - NAT traversal or reachable ingress,
   - port availability for allowed ports,
   - baseline bandwidth class.

Step 1 — Nominate capacity for Collective windows.

1. In `Provider Portal`, nominate for SLA=`Collective`.
2. Decide whether to opt into **session roles**:

- `member-only` (can host any rank),
- `leader-capable` (can host rank 0 / rendezvous responsibilities).

Step 2 — Gate close assignment and membership proof.

1. At $T_0 - 5$m, agent receives:
   - its assigned ranks (package descriptors with session fields),
   - session membership list hash,
   - overlay bootstrap material (ephemeral certs/tokens).
2. Agent verifies:
   - package `image_digest` and input roots,
   - membership proof matches session hash,
   - allowed endpoints list.

Step 3 — Prepare window actions (plus overlay join).

1. Run Standard prepare steps (preflight, pull image, prefetch).
2. Join overlay session:
   - establish tunnels to rendezvous/peers as specified,
   - enforce members-only routing (drop all non-member traffic),
   - run handshake and key exchange,
   - report `POST /sessions/{sid}/ready`.
3. If overlay join fails before T_0:
   - agent marks itself not-ready,
   - scheduler may reassign rank to standby (policy),
   - repeated failures degrade provider reputation.

Step 4 — Execute collective worker and report.

1. Start worker at T_0 with `rank` and `world_size`.
2. Emit first heartbeat + first batch within ≤ 2m.
3. Collect and submit:
   - output_root and checkpoint hashes,
   - signed net telemetry root (RTT/bandwidth measurements),
   - raw GPU time and logs URI.

Step 5 — QC, certificate verification, and settlement.

1. QC may verify certificates (e.g., $Mv = 0$) and spot-check telemetry integrity.

2. If net_SLO violated persistently:
   - session may be marked Degraded,
   - partial slashing or reputation penalty may apply even if compute was delivered.
3. Settlement pays accepted packages by `paid_ngh`; vouchers consumed accordingly.

7.5 Multi-Provider Example: "One Buyer, Many Sellers" (Concrete End-to-End)

Scenario. Buyer runs CADO-NFS with: (i) 120 Standard sieve packages (distributed across 10 providers), (ii) 32 Collective LA worker packages (distributed across 6 providers).

1. Buyer buys exposure:
   - Standard key tuple for sieve NGH,
   - Collective key tuple for LA NGH,
   - deposits both to one job.
2. Providers nominate:
   - providers A–J nominate Standard Tier 1 capacity,
   - providers B, D, F, G, H, I nominate Collective Tier 1 capacity.
3. Scheduler finalizes:
   - sieve packages spread widely (independent),
   - LA ranks placed with network feasibility (measured inter-provider connectivity),
   - standby ranks reserved for both modes.
4. Prepare:
   - Standard providers pull image + prefetch,
   - Collective providers additionally join overlay and attest session-ready.
5. Execute:
   - sieve outputs stream in as relation shards,
   - LA session runs block Lanczos; checkpoint and kernel vector produced.
6. Verify:
   - Standard packages: dup/canary/spot,
   - LA: certificate verification of kernel vector ($Mv = 0$) + telemetry spot checks.
7. Settle:
   - accepted paid_ngh summed per provider,
   - vouchers consumed by key tuple,
   - slashes fund supplier-of-last-resort and neutral pool.

8. Buyer exits:
    - downloads receipt bundle,
    - replays proofs against on-chain roots,
    - reproduces factorization result (gcd verification).

8 Testbed Execution Plan (4× RTX 3060): How We Test v2 End-to-End

This section specifies a concrete, repeatable testing methodology using a small local GPU cluster: either (i) four discrete RTX 3060 GPUs, or (ii) one machine that exposes four RTX 3060 devices. The purpose is not to claim H100-scale performance, but to validate *the full v2 mechanism*: futures → vouchers → deposit → gate close → prepare → execution → QC → on-chain anchoring → settlement, including the Collective SLA overlay.

8.1 Testing Principles (What Must Be True)

- **Mechanism over magnitude.** We test correctness of protocol flow, not absolute throughput.
- **Multi-provider behavior.** Even on one machine, we model decentralization by running multiple provider identities.
- **Reproducible runs.** Each test has a deterministic seed, fixed package manifests, and produces the same receipt roots.
- **Two modes.** We test (A) Standard SLA (many independent packages) and (B) Collective SLA (a small session).

8.2 Testbed Parameterization (RTX 3060 Defaults)

We define a **DemoNet** parameterization that preserves v2 structure but rescales constants to fit a 3060 cluster.

- **Tier mapping:** RTX 3060 capacity is classified as **Tier 3**.
- **Normalization baseline (DemoNet NGH).** For the demo, we define `demo_NGH` as "one hour on a reference RTX 3060 under the published microbenchmark profile." All receipts and settlements in the demo are denominated in `demo_NGH`. Production NGH remains defined on H100.
- **Package target size:** target **0.25–1.0 demo_NGH per package** (aim for 5–20 minutes wall time).
- **Collective world size:** $W \in \{2, 4\}$ ranks (bounded by available GPUs).
- **QC rates (DemoNet):** preserve mechanisms, but ensure feasibility on 4 GPUs:
    - Duplication: **at least 1 duplicated package per 20 packages** (or reserve 1 GPU for QC if possible).
    - Canaries: keep at ∼1% items (cheap).
    - Spot re-exec: **at least 1 spot-check per 20 packages** on the neutral pool.

- **Timing constants:** keep v2 timing constants (gate close $T_0 - 5$m, prepare $T_0 - 3$m, first output ≤ 2m). If CADO kernels cannot emit output that quickly, implement **heartbeat** as "first output" (still within 2 minutes).

8.3 Local Multi-Provider Emulation (Required)

To test decentralized scheduling and settlement, we run **four provider agents** with **four distinct identities**, even if all GPUs are physically local.

- **Provider identities:** `provA`, `provB`, `provC`, `provD` each have:
  - unique provider keys / fingerprint,
  - their own agent process,
  - an exclusive CUDA device binding: `CUDA_VISIBLE_DEVICES = 0,1,2,3` respectively.
- **Nomination separation:** each provider independently nominates capacity per window (even if all capacity is local).
- **Overlay realism:** to emulate cross-provider network variance, we optionally inject latency/jitter/bandwidth constraints between provider processes using OS-level traffic control (e.g., `tc netem`). This produces credible "Collective SLA" telemetry.

8.4 Software Components Needed for the Testbed

A minimal test deployment consists of:

1. **Market/clearing service (demo):** creates physically deliverable positions and mints demo vouchers at day-ahead.
2. **Voucher contracts (or simulated chain):** Voucher1155, ReceiptRegistry, Settlement.
3. **Scheduler:** accepts nominations, computes placements, enforces gate close, issues lots.
4. **Artifact Relay:** serves package manifests and inputs over HTTP with checksums.
5. **Provider agent:** pulls image, prepares, runs, emits receipts, joins overlay for collective sessions.
6. **QC oracle + neutral pool runner:** runs duplicates/spot checks and anchors receipt/qc roots.
7. **Buyer CLI:** builds packages, submits jobs/sessions, deposits vouchers, downloads receipts.

8.5 Test A: Standard SLA Smoke Test (Independent Packages)

Objective. Validate end-to-end flow for embarrassingly parallel GPU work across multiple providers: vouchers, deposit, gate close, prepare, execution, QC, on-chain anchoring, settlement.

8.5.1 A1. Setup (One-Time per Test Run)

1. Choose window key tuple: (region=`local`, hour=`next_hour`, SLA=`Standard`, tier=`T3`).
2. Buyer creates a small job:
   - package count: 20 packages,
   - target size: 0.5 demo_NGH each,
   - determinism: `bitexact` if possible, else `fp_tolerant`.
3. Insert canaries:
   - ∼1% known-answer items.
4. Upload artifacts to Relay (manifest + canaries + any small inputs).
5. Buyer acquires a physically deliverable position and triggers voucher mint at demo day-ahead/expiry.
6. Buyer deposits vouchers to job (total demo_NGH + buffer).

8.5.2 A2. Provider Actions (Per Window)

1. Each provider nominates: e.g., 3 demo_NGH for the window.
2. At $T_0 - 5$m, scheduler assigns lots and publishes placements.
3. At $T_0 - 3$m..T_0, providers:
   - pull image digest,
   - prefetch first input chunk,
   - attest `/prepare_ready`.
4. At T_0, providers run packages and emit first output or heartbeat ≤ 2m.
5. Providers submit `/result` including output roots and raw GPU time.

8.5.3 A3. QC & Settlement Checks (Assertions)

1. QC selects:
   - at least 1 duplicated package (run on a second provider),
   - at least 1 spot re-exec on neutral pool.
2. Acceptance logic:
   - canaries pass for accepted packages,
   - duplicated package matches (bitexact or within tolerance),
   - spot check matches.
3. QC oracle computes `receipt_root` and `qc_root` and anchors them on-chain.
4. Settlement consumes vouchers 1:1 with accepted demo_NGH and pays providers by `paid_ngh`.
5. Buyer downloads receipt bundle and verifies Merkle proofs against on-chain roots.

8.5.4 A4. Pass/Fail Criteria

- **Pass** if: no package violates prepare window; $\geq 95\%$ emit first output/heartbeat within 2 minutes; QC produces consistent accept/reject; settlement consumes correct voucher keys; receipts verify against roots.
- **Fail** if: any accepted receipt cannot be proven in `receipt_root`; vouchers are consumed incorrectly; or a provider is paid for a rejected package.

8.6 Test B: Collective SLA Smoke Test (4-Rank Session)

Objective. Validate overlay membership, network telemetry, start skew monitoring, and collective execution with 2–4 ranks.

8.6.1 B1. Session Configuration

1. Choose window key tuple: (region=`local`, hour=`next_hour`, SLA=`Collective`, tier=`T3`).
2. Define session:
   - world_size: $W = 4$ (or $W = 2$ if fewer GPUs),
   - membership list: 4 package_ids,
   - net profile: `collective_v1_demo` with modest targets (e.g., RTT p95 $\leq$ 20ms local).
3. Buyer creates 4 collective worker packages:
   - each has `rank`, `world_size`, `session_id`,
   - each emits heartbeat within 2 minutes, and writes a deterministic checkpoint hash.
4. Buyer acquires/delivers Collective vouchers and deposits to the job.

8.6.2 B2. Overlay Emulation (Optional but Recommended)

To mimic cross-provider behavior even on one machine, apply artificial network constraints:

- Example: add 30ms RTT and 200 Mbps cap between `provA` and `provD`.
- Example: add jitter (5ms) between `provB` and `provC`.

This forces telemetry to show non-trivial conditions and validates net_SLO enforcement.

8.6.3 B3. Execution & Assertions

1. At gate close, scheduler pins membership and assigns ranks to providers (may be 1 rank per provider).
2. During prepare, each provider:
   - completes Standard prepare,
   - joins overlay, completes handshake,
   - attests `/sessions/{sid}/ready`.
3. At T_0, session becomes `Active` only if all ranks are ready.

4. Each rank runs a simple collective workload:
   - e.g., all-reduce on random tensors (demo-only), or CADO LA micro-kernel (preferred).
5. Providers submit:
   - output_root (checkpoint hashes),
   - net telemetry root (signed RTT/bandwidth samples),
   - raw GPU time.
6. QC:
   - spot-checks telemetry integrity,
   - verifies deterministic checkpoint hashes or certificate if using LA kernel.

8.6.4 B4. Pass/Fail Criteria

- **Pass** if: overlay membership is enforced (no non-member connectivity), start-skew p95 meets demo target, telemetry is anchored, and settlement consumes Collective vouchers correctly.
- **Fail** if: any rank can send/receive packets to non-members, or if session readiness cannot be proven in receipts/QC root.

8.7 Test C: CADO-NFS Demo (Scaled for RTX 3060)

Objective. Demonstrate the narrative goal: CADO-NFS runs across GPUs from multiple "providers" and is delivered via futures/vouchers.

8.7.1 C1. Scaled Parameters

- Choose a small-to-medium N and CADO parameter set that completes within a few hours on 4×3060.
- Split into:
  - sieve packages (Standard SLA): many short intervals,
  - linear algebra packages (Collective SLA): $W = 2$ or $W = 4$ ranks.

8.7.2 C2. Demonstration Checklist

1. Buyer buys physically deliverable exposure for both Standard and Collective keys (or just Collective if unified).
2. Vouchers mint day-ahead; buyer deposits to the job.
3. Multiple providers nominate and receive placements.
4. Sieve stage outputs relation shards with canonical hashes; QC catches at least one duplicate.
5. LA stage runs as a collective session; outputs include kernel vector/checkpoint hashes.
6. QC verifies algebraic certificate when applicable (e.g., verify $Mv = 0$) and anchors roots.
7. Settlement pays providers and consumes vouchers 1:1 with accepted demo_NGH.
8. Final factor is verified (gcd check) and included as demo artifact.

8.8 Negative Tests (Required)

These tests ensure the system fails safely and proves slashing/assignment logic.

1. **Prepare miss.** Force one provider to skip `/prepare_ready`; verify reassignment and no billing during prepare.
2. **First-output miss.** Delay output beyond 2 minutes; verify cancel + reassignment.
3. **Bad output.** Corrupt outputs for a duplicated package; verify QC rejection + slashing.
4. **Overlay ACL breach (Collective).** Attempt to connect from a non-member process; verify packet drop + logged violation.
5. **Voucher mismatch.** Deposit wrong-key vouchers; verify settlement refuses consumption and spot-settles shortfall.

8.9 Artifacts to Collect for the Final Report (What We Show)

For each test run, we archive:

- package index, manifests, canary file hashes, and image digests,
- nomination logs and gate-close placement mapping,
- per-package receipts and QC outcomes,
- anchored `receipt_root` and `qc_root` and Merkle proofs for a sample receipt,
- settlement summary: vouchers consumed, paid_ngh per provider, slashes (if any),
- overlay telemetry plots (start skew, RTT, bandwidth) for Collective tests,
- final CADO factor output (and verification).

9 Conclusion

Version 2 preserves the v1 financial-to-physical bridge (futures/forwards → vouchers → delivery → on-chain anchoring), keeps package-scoped receipts and mechanical settlement, and adds one capability: controlled multi-provider collective connectivity. CADO-NFS provides a public, deterministic, certificate-verifiable workload to demonstrate that GPUs from multiple providers can be composed into one logical job under hourly delivery contracts.

10 Glossary & Concept Dictionary (Read This First)

This section is a deliberately non-technical dictionary for the rest of the document. It defines every recurring term in plain language, explains why it exists, and then (only after that) gives the precise meaning used by the platform. Readers should use this as the primary reference when a later section feels jargon-heavy.

10.1 How to use this glossary

Each entry is written in the same structure:

1. **Plain meaning:** an intuitive description (no jargon).
2. **Why we need it:** what problem this concept solves in the platform.
3. **Precise meaning in v2:** the strict definition used by the protocol.
4. **Where it appears:** the main sections that rely on it.

10.2 Core Building Blocks

Compute Marketplace

1. **Plain meaning:** a place where people buy and sell GPU time like a commodity.
2. **Why we need it:** to turn "finding GPUs" into a liquid market with transparent prices.
3. **Precise meaning in v2:** the combination of (i) a financial layer that trades standardized compute contracts and (ii) a delivery layer that schedules, runs, verifies, and settles real GPU work.
4. **Where it appears:** §1, §2, §3, §6.

Hour Window

1. **Plain meaning:** a one-hour reservation slot, like 14:00–15:00.
2. **Why we need it:** it makes compute tradable (standard unit), schedulable (start time), and settleable (clear accounting).
3. **Precise meaning in v2:** the window $[T_0, T_0 + 1\text{h}]$; the product key includes this window.
4. **Where it appears:** §2 (everywhere), §3 (timeline), §6 (settlement).

T_0 ("T-zero")

1. **Plain meaning:** the exact start time of your chosen hour window.
2. **Why we need it:** all timing rules (prepare, first-output, deadline) are measured relative to one unambiguous start time.
3. **Precise meaning in v2:** the start timestamp of the hour window. Prepare and gate close are defined as $T_0 - 3\text{m}$ and $T_0 - 5\text{m}$.
4. **Where it appears:** §2.1, §2.4, §3.

Region

1. **Plain meaning:** where the GPUs are located (geographic cluster).
2. **Why we need it:** latency, network reliability, and supply differ by geography; pricing should reflect that.
3. **Precise meaning in v2:** a discrete label (e.g., `us-east`) that is part of the product key and constrains scheduling.
4. **Where it appears:** §2.2, §2.4, §6.

Capability Tier (Tier 1/2/3)

1. **Plain meaning:** a bucket of GPU "power level" and memory.
2. **Why we need it:** we cannot price an H100 hour the same as a 3090 hour; tiers keep products comparable.
3. **Precise meaning in v2:** a discrete tier label (T1/T2/T3) with published constraints (VRAM range, expected throughput class).
4. **Where it appears:** §2.2, §2.3, P1–P6.

SLA (Service Level Agreement) / Service Class

1. **Plain meaning:** the "rulebook" for how reliably and how quickly your work starts/finishes.
2. **Why we need it:** buyers pay for reliability; providers get paid more for stricter guarantees.
3. **Precise meaning in v2:** two classes: `Standard` and `Collective`. Both share the hour window but Collective adds guaranteed in-session connectivity via an overlay network and network SLOs.
4. **Where it appears:** §2.1, §2.4, P3, P5, D4, D8.

Prepare Window

1. **Plain meaning:** the short warm-up period right before the hour starts.
2. **Why we need it:** to avoid slow cold starts at T_0 (pulling containers and fetching inputs after the clock starts).
3. **Precise meaning in v2:** $[T_0 - 3\text{m}, T_0]$. Provider must preflight hardware, pull exact image digest, prefetch first input chunk, and attest readiness. This time is not billed.
4. **Where it appears:** §2.1, §2.4, D8.

First Outputs / First Heartbeat

1. **Plain meaning:** an early sign the run is actually working, not stuck.
2. **Why we need it:** if a provider is failing, we need to know fast enough to reassign and still hit the hour deadline.
3. **Precise meaning in v2:** each package must emit first outputs within 2 minutes of T_0. For Collective SLA long-running kernels, an equivalent "first heartbeat + first batch" signal is acceptable.
4. **Where it appears:** §2.1, §2.4, P1, D4.

NGH (Normalized GPU-Hour)

1. **Plain meaning:** a standardized way to count GPU time so different GPUs can be compared.
2. **Why we need it:** otherwise providers with slower GPUs would be unfairly paid, and buyers couldn't compare prices.
3. **Precise meaning in v2:** one hour of H100-class baseline performance. Raw GPU hours are converted using a multiplier q derived from benchmarks: $\texttt{paid_ngh} = \texttt{gpu_time_hours_raw} \cdot q$.
4. **Where it appears:** §2.1, §2.2, P4, D1.

Voucher

1. **Plain meaning:** a digital claim ticket that says "I own X normalized GPU-hours during this hour window."
2. **Why we need it:** to separate pricing risk (buy/sell/hedge) from execution risk (delivery), and to make compute liquid.
3. **Precise meaning in v2:** an on-chain token keyed by (region, hour, SLA, tier) that is minted at forward/futures settlement, can be traded, can be deposited to a job, and is consumed 1:1 with accepted NGH.
4. **Where it appears:** §2.2, §2.3, D1, D9.

Physically Deliverable Futures / Forwards

1. **Plain meaning:** a contract you buy now that later gives you vouchers (real compute claims), not just cash.
2. **Why we need it:** to let buyers lock in compute prices in advance (hedging), like commodities markets.
3. **Precise meaning in v2:** a contract that settles into vouchers of the exact key tuple at expiry/day-ahead.
4. **Where it appears:** §1, §2.4, P6, D9.

Spot Index

1. **Plain meaning:** the published "going price" for compute for a given hour/tier/SLA.
2. **Why we need it:** to settle shortages, price spot clearing, and value positions fairly.
3. **Precise meaning in v2:** an hourly VWAP-like statistic computed from accepted work and/or voucher trades, per key tuple, with manipulation guardrails.
4. **Where it appears:** §2.2, P6, D9.

10.3 Execution Objects

Job

1. **Plain meaning:** everything the buyer wants done (possibly many packages).
2. **Why we need it:** it's the unit the buyer thinks in; it ties packages, vouchers, and receipts together.
3. **Precise meaning in v2:** a job_id referencing a package index and optional collective sessions; can span many providers.
4. **Where it appears:** §2.4, §3, D7, D10.

Package (Sealed Package)

1. **Plain meaning:** a single self-contained "task" that can run on one GPU resource.
2. **Why we need it:** it's the atomic unit for scheduling, verification, and settlement. Without an atomic unit, disputes and payment are ambiguous.
3. **Precise meaning in v2:** a PackageDescriptor: immutable container digest + command + input root + output schema + rules for comparison. Platform never edits or splits a package. It runs on one provider resource note.
4. **Where it appears:** §2.1, §2.3, §2.4, P1–P5, D4–D8.

Lot (Package as Scheduled Unit)

1. **Plain meaning:** a package after the scheduler has assigned it to a provider and hour.
2. **Why we need it:** to talk about a scheduled execution instance (with a provider and a window).
3. **Precise meaning in v2:** the scheduler's internal identifier for a package execution attempt within an hour, used by provider APIs.
4. **Where it appears:** §2.4 (APIs), D4, D8.

Provider (Seller of Compute)

1. **Plain meaning:** a person/company running GPUs who wants to earn money by renting them out.
2. **Why we need it:** the supply side of the market; decentralization means many independent providers can participate.
3. **Precise meaning in v2:** an identity with a provider_fingerprint that nominates capacity, receives lots, runs packages, and submits receipts/telemetry.
4. **Where it appears:** §2.4, D2, D8.

Nomination

1. **Plain meaning:** a provider saying "I can supply this much compute during this hour."
2. **Why we need it:** the scheduler needs to know what supply exists before placing jobs.
3. **Precise meaning in v2:** a signed declaration of deliverable NGH for a key tuple and hour, submitted ahead of gate close.
4. **Where it appears:** §2.4, D2, D4.

Gate Close

1. **Plain meaning:** the moment when assignments become final and no one can change them.
2. **Why we need it:** to give providers time to prepare and to stop last-second reshuffling that would break on-time starts.
3. **Precise meaning in v2:** $T_0 - 5\text{m}$. After this time, package→provider mappings and session membership are fixed.
4. **Where it appears:** §2.4, P3, D4.

10.4 Verification & Trust

QC (Quality Control)

1. **Plain meaning:** the platform's built-in checks that providers did the work correctly.
2. **Why we need it:** without verification, cheating is profitable and buyers can't trust results.
3. **Precise meaning in v2:** a fixed sampling policy: 5% duplication, ∼1% canaries, 0.5% spot re-exec. Decisions are package-scoped and binary.
4. **Where it appears:** §2.1, §2.4, P2, P5, D5.

Duplication

1. **Plain meaning:** run the same package twice on different providers and compare answers.
2. **Why we need it:** it's the cleanest way to detect wrong computation.
3. **Precise meaning in v2:** 5% of packages are duplicated; comparisons use bit-exact or FP-tolerant policy.
4. **Where it appears:** §2.4, P2, D5.

Canary

1. **Plain meaning:** a small set of test inputs with known correct outputs.
2. **Why we need it:** a cheap way to catch obvious cheating or broken environments.
3. **Precise meaning in v2:** about 1% of each package's items are canaries inserted by the buyer (or generated from a known corpus).
4. **Where it appears:** §2.4, P2, D7.

Spot Re-execution (Neutral Pool)

1. **Plain meaning:** randomly rerun a small part of someone's work on trusted hardware.
2. **Why we need it:** to check non-duplicated packages without duplicating everything.
3. **Precise meaning in v2:** rerun 0.5% of items on an independent neutral pool; on dispute, rerun 10%.
4. **Where it appears:** §2.4, P5, D5, D11.

Certificate Verification (Algebraic Certificate)

1. **Plain meaning:** a short proof that the result is correct (faster than rerunning the whole job).
2. **Why we need it:** collectives are expensive to duplicate; certificates can verify correctness cheaply.
3. **Precise meaning in v2:** an optional workload-specific check (e.g., in CADO: verify $Mv = 0$ and that the factor is valid). Recorded as a QC signal; used in acceptance for certificate-enabled schemas.
4. **Where it appears:** §4, §2.4, P5, D5.

Receipt

1. **Plain meaning:** the platform's signed record of what was run, on what provider, and what the outputs were.
2. **Why we need it:** it's the bridge between execution and payment; it's also how audits work later.
3. **Precise meaning in v2:** per-package record: provider_fingerprint, input/output roots, raw GPU time, multiplier q, paid_ngh, QC flags, and optional net telemetry root for Collective packages.
4. **Where it appears:** §2.3, §2.4, D10.

Merkle Root / Anchoring

1. **Plain meaning:** a cryptographic fingerprint of a big list of things (receipts/QC events) that can be verified later.
2. **Why we need it:** so the chain stores a small commitment, while anyone can later prove a particular receipt was included.
3. **Precise meaning in v2:** `receipt_root` and `qc_root` posted on-chain after the hour. Receipt bundles include Merkle proofs.
4. **Where it appears:** §2.4, D1, D10.

Settlement

1. **Plain meaning:** paying providers and consuming the buyer's vouchers based on what was actually accepted.
2. **Why we need it:** to ensure the financial contract and physical delivery are consistent.
3. **Precise meaning in v2:** pay per accepted package by `paid_ngh`; consume deposited vouchers 1:1 with accepted NGH; shortfalls settle at the spot index.
4. **Where it appears:** §2.4, P6, D1.

Slashing

1. **Plain meaning:** a penalty for failing or cheating (losing some payment or stake).
2. **Why we need it:** to make cheating unprofitable and fund backup capacity.
3. **Precise meaning in v2:** QC failures and missed milestones forfeit payout and may impose an imbalance fee; the pool funds neutral QC and standby.
4. **Where it appears:** §2.4, D1, D5, D11.

10.5 Connectivity & Collective Terms (v2 additions)

Collective (Collective SLA)

1. **Plain meaning:** a distributed job step where multiple workers need to talk to each other while running.
2. **Why we need it:** some GPU jobs require synchronization (reductions, sharded computations, distributed LA).
3. **Precise meaning in v2:** an SLA class that guarantees a session-scoped overlay network among a fixed membership of packages, possibly spanning multiple providers, with published network SLOs.
4. **Where it appears:** §2.1, §2.3, §2.4, P3, P5.

Collective Session

1. **Plain meaning:** the "room" a set of distributed workers join so they can talk only to each other.
2. **Why we need it:** to safely provide peer connectivity on a decentralized platform without opening the internet.
3. **Precise meaning in v2:** a session_id with a fixed world_size and membership list finalized at gate close; overlay expires at hour end.
4. **Where it appears:** §2.3, §2.4, D4, D8.

Rank / World Size

1. **Plain meaning:** rank is a worker's ID (0,1,2,...) and world size is how many workers exist total.
2. **Why we need it:** distributed programs need consistent identities to coordinate.
3. **Precise meaning in v2:** rank $\in \{0, \ldots, W-1\}$ where W is world_size in the session descriptor; fixed at gate close.
4. **Where it appears:** §2.3, Buyer Walkthrough B, D4.

Overlay Network

1. **Plain meaning:** a private, temporary network connecting session members.
2. **Why we need it:** providers are strangers; we must restrict connectivity to session members only.
3. **Precise meaning in v2:** session-scoped encrypted tunnels with members-only ACL and deny-by-default routing; teardown at hour end.
4. **Where it appears:** §2.3, §2.4, D8.

Network SLO (Start Skew / RTT / Bandwidth)

1. **Plain meaning:** performance targets for how well session members can communicate.
2. **Why we need it:** distributed work fails if networking is too slow or too inconsistent.
3. **Precise meaning in v2:** fixed targets attached to Collective SLA, measured via signed telemetry: start skew p95, RTT p95, bandwidth p50.
4. **Where it appears:** §2.7, P3, D4, D8, D10.

References

(Include CADO-NFS references, NFS background, block Lanczos/Wiedemann references, and any benchmarking methodology references.)